

Descriere in limbaj natural

Tuesday, May 19, 2026 7:34 PM

Programul = cod structurat care efectueaza niste operatii/pasi pentru a rezolva o problema, astfel incat sa acoperim toate cerintele, restrictiile etc.

Descrierea in limbaj natural reprezinta explicarea codului pe intelesul tuturor, practic se raspunde la intrebari de genul: *Ce face?*, *De ce ai folosit ...?*, *De ce este eficient din punct de vedere a timpului de executiei/ memoriei utilizate?* Etc.

Ex de asa da: Programul citeste 2 numere intregi de la tastatura, efectueaza suma acestora si afiseaza (in consola) rezultatul.

Ex de asa nu: Se citeste a si b, se afiseaza a+b.

Good practice:

1. Variabile cu nume sugestive. Ex: anterior/prev/previous
NU a X
2. Comentarii in cod (blocuri repetitive, blocuri decizionale, in functii pt a descrie ce face functia, care sunt parametrii, ce returneaza, daca returneaza ceva)

Eficienta unui program:

1. Eficienta in timp

- Reprezinta cat timp ii ia codului sa ruleze, adica cati pasi efectueaza pana cand se incheie programul

Pentru atribuiuri avem complexitate $O(1)$, deoarece este un singur pas de executat, nu tine cont de cat de mare este numarul sau cat de lung este un sir de caractere etc. Daca avem numai atribuiuri complexitatea este $O(1)$, adica se efectueaza foarte rapid.

$x=5$ sau $x=500000$ are aceeasi complexitate in timp, adica se efectueaza dintr-un singur pas

Pentru bucle simple, de sine statatoare, care au un numar de pasi de la 1 la n, sau pana la un punct, complexitatea este $O(n)$ - liniara, deoarece se numara cati pasi se fac la fiecare incrementare/decrementare/schimbare a contorului. Se executa dintr-o singura parcurgere.

```
Ex:
s=0;
p=1;
for(int i=1; i<=n;i++)
{
    s=s+i;
    p=p*i;
}
```

Complexitatea noastra este $O(n)$, adica liniara, deoarece avem 2 atribuiuri pt s si p, deci 2pasi, bucla for are n pasi * 2 = 2 * n pasi $\Rightarrow O(2+2*n)$, dar adunarea si inmultirea cu constante nu se pune $\Rightarrow O(n)$

Pentru bucle in bucle, complexitatea este $O(n^2)$ sau $O(n*m)$, unde n este numar de pasi pt prima bucla si m nr de pasi pt a doua bucla. Numita complexitate patratica.

Daca avem 2 sau mai multe bucle una dupa alta complexitatea este liniara, adica $O(n)$.

Daca avem 2 sau mai multe bucle una in alta avem complexitate polinomiala (ne gandim la puteri) - $O(n^{nr_bucle})$ sau $O(n_1 * n_2 * ... * n_{nr_bucle})$. De obicei se foloseste intr-un mod mai simplificator

$O(n^{nr_bucle})$.

Pentru cazuri in care domeniul de cautare se injumatateste la fiecare pas, complexitatea este logaritmica si se noteaza $O(\log n)$. Ex: cautarea binara

Scara de eficienta de la cel mai eficient la cel mai putin:

$O(1)$

$O(\log n)$

$O(n)$

$O(n * \log n)$

$O(n^{nr_bucle})$

Ne dorim sa fim cu eficienta undeva intre logaritmica, liniara sau inmultire intre cele 2.

!La sortari normale complexitatea este $O(n^2)$, fiindca avem bucla in bucla, parcurgerea de 2 ori a elementelor, iar la cele eficiente este $O(n \log n)$, deoarece suntem obligati sa parcurgem macar o data sirul de numere de sortat.

2. Eficienta in spatiu

- Reprezinta cat spatiu de memorie consuma programul meu, adica fiecare variabila are un spatiu atribuit in RAM. Putem vizualiza asta ca pe o casuta.
- Variabilele simple au un spatiu bine definit de tipul acestora

Tip de date	Bytes	Biti
Bool, char	1	8
Int, float	4	32
Long long, double	8	64

- Tablourile au spatiul ocupat egal cu dimensiunea*tipul_de_date. Pt vectori $nr_elemente * tip_de_date$, pt matrici $linii * coloane * tip_de_date$.
- Cand avem numar foarte mare de date de input, nu folosim vectori, incercam sa rezolvam problema inca de la citirea numerelor

Formulare pt subpuncte de genul date la bac/teste.

Algoritmul la subpunctul x este eficient, deoarece este **liniar**/logaritm si foloseste variabile simple/vectori de frecventa/caracteristici (**de tip bool**). Cu alte cuvinte, din punct de vedere a timpului de executie/executare are complexitate $O(n)/O(\log n)/O(n \log n)$, deoarece operatiile din program se executa odata cu citirea de la tastura/din fisier a elementelor. De asemenea, este eficient din punct de vedere a spatiului de stocare/ memoriei utilizate, fiindca foloseste variabile simple/vectori de frecventa/caracteristici (**de tip bool**), *in total folosind pe stocarea variabilelor declarate x bytes (=8*x biti).*

Programul implementeaza ... (ce face?, ce fac variabilele / la ce se folosesc, de ce am ales anumite verificari etc.)